



Sistemi Distribuiti

Laurea in Informatica

Michele Ciavotta

michele.ciavotta@unimib.it

Programmazione in JavaScript

1. **Le basi di JavaScript** · Evoluzione del linguaggio · var, let, const
2. **Funzioni e tipi** · Forme sintattiche · Tipo String e metodi
3. **Array e classi** · Metodi sugli array · Classi · Iterazione
4. **Programmazione funzionale** · forEach · map · filter · reduce
5. **Oggetti e riferimenti** · Esempi pratici · Riferimenti bibliografici

SEZIONE 1

LE BASI DI JAVASCRIPT

Evoluzione di JavaScript

ECMAScript è il nome ufficiale dello standard

Gli standard più diffusi e importanti utilizzati nell'industria del software sono

- ES5 (alias ECMAScript 2009)
- ES6 (alias ECMAScript 2015)

Edizione	Anno
ECMAScript 1	1997
ECMAScript 2	1998
ECMAScript 3	1999
ECMAScript 4	abbandonata
ECMAScript 5	2009
ECMAScript 5.1	2011
ECMAScript 2015 (ES6)	2015
ECMAScript 2016	2016
ECMAScript 2017	2017
ECMAScript 2018	2018
ECMAScript 2019	2019
ECMAScript 2020	2020
ECMAScript 2021	2021
ECMAScript 2022	2022
ECMAScript 2023	2023
ECMAScript 2024	2024
ECMAScript 2025	2025

<https://tc39.es/ecma262/>

var, let e const

In ES5 le variabili sono dichiarate con **var**: hanno **function scope** (visibili in tutta la funzione che le contiene) e **non rispettano lo scope di blocco**

ES6 ha introdotto due nuove parole chiave per dichiarare variabili.

- **let** dichiara una variabile con **block scope**: vale solo nel blocco { ... } in cui è dichiarata
- **const** ha block scope come **let** e in più **non può essere riassegnata** dopo l'inizializzazione (il contenuto di oggetti e array resta però mutabile)

```
var x = 10; // qui x vale 10
{
  let x = 2; // qui x vale 2
  let y = 5; // qui y vale 5
  var z = 8; // qui z vale 8
}

// qui x vale 10
// y NON è accessibile qui
// z PUÒ essere usata qui (var ignora il blocco)
```

```
const PI = 3.14;
PI = 3.14159; // ERRORE: riassegnazione non consentita
const arr = [1, 2, 3];
arr.push(4); // OK: il contenuto può essere modificato
```

Buona regola: usare **var** solo per le variabili globali,
let e **const** in tutti gli altri casi

var, let e const

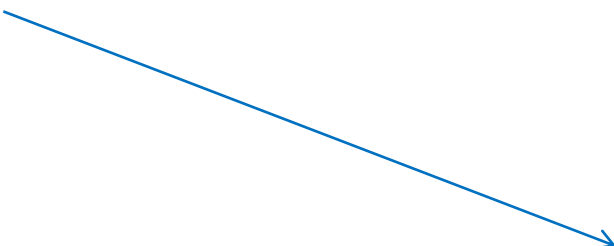
La **const** permette di definire variabili **let** con un valore costante (cioè che non può essere modificato) (block scope)

ATTENZIONE: le **const** sono contenitori costanti, ma i valori contenuti possono cambiare

- Le variabili semplici sono immutabili
- I valori delle variabili composte possono cambiare



```
1. const ages = [15, 35, 28, 18];
2.    // Here ages is 15, 35, 28, 18
3. for (i = 0; i < ages.length; i++) {
4.     ages[i] = ages[i] + 1;
5. }
6.    // Here ages is 16,36,29,19
7. ages[4] = 56;
8.    // Here ages is 16,36,29,19,56
9. ages = [43, 26]; // ERROR!
```



```
1. var x = 10;
2.    // Here x is 10
3. {
4.     const x = 2; // Here x is 2
5.     x = 4; // ERROR!
6. }
7.    // Here x is 10
```

Buona regola: usare **const** in tutti i casi possibili

Controllo di flusso

Strutture standard

- **if/else, switch, for, while, do...while**: identici a Java

Iteratori specifici di JS

- **for...of** → itera sui **valori** (array, stringhe, ecc.)
- **for...in** → itera sulle **chiavi** di un oggetto (o indici di array)
- Le condizioni accettano valori **truthy/falsy** (es. 0, "", null, undefined → falsy)

```
const arr = [10, 20, 30];
```

```
// for classico  
for (let i = 0; i < arr.length; i++) {  
  console.log(arr[i]);  
}
```

```
// itera sui valori  
for (const v of arr) {  
  console.log(v); // 10, 20, 30  
}
```

```
// itera sugli indici/chiavi  
for (const k in arr) {  
  console.log(k); // 0, 1, 2  
}
```

SEZIONE 2

Funzioni e tipi

Le diverse forme sintattiche per le funzioni

- ES5 definisce due modi per definire le funzioni

```
// ES5
function f(x, y) {
  return x * y;
}

var f = function(x, y) { // let è stata introdotta in ES6
  return x * y;
}
```

- ES6 ha introdotto una nuova sintassi per definire le funzioni: *arrow function*

```
// ES6
const f = (x, y) => x * y;
```

- La keyword `return` e le parentesi graffe `{ }` possono essere omesse solo quando la funzione ha una sola istruzione. Per questa ragione è buona abitudine usarle sempre

```
const f = (x, y) => { return x * y };
```

https://www.w3schools.com/js/js_arrow_function.asp

Il tipo String (manipolazione testo)

- Una stringa JavaScript è una sequenza di zero o più caratteri racchiusi tra virgolette.

```
1.let carName1 = "Volvo XC60"; // virgolette doppie  
2.let carName2 = 'Volvo XC60'; // virgolette singole
```

- Per ottenere la lunghezza di una stringa, si usa la proprietà `length` integrata.

```
3.let sln = carName1.length; // = 10
```

- L'operatore `+` può essere usato per concatenare le stringhe.

```
4.let txt1 = "John";  
5.let txt2 = 'Doe';  
6.let txt3 = txt1 + " " + txt2; // John Doe
```

- Sommando un numero e una stringa, il risultato sarà una stringa!

```
4.let z = "John" + 35; // John35
```

Il tipo String: metodi (1)

- Il metodo `indexOf()` restituisce l'indice (la posizione) della prima occorrenza di un testo specifico in una stringa

```
1. const str = "Please locate where 'locate' occurs!";
```

```
2. let pos = str.indexOf("locate"); // 7
```

- Il metodo `lastIndexOf()` restituisce l'indice dell'**ultima** occorrenza di un testo specifico in una stringa

```
3. let pos = str.lastIndexOf("locate"); // 21
```

- Sia `indexOf()` sia `lastIndexOf()` restituiscono -1 se il testo non viene trovato

- Entrambi i metodi accettano un secondo parametro che indica la posizione di partenza per la ricerca

```
4. let pos = str.indexOf("locate", 15); // 21
```

Il tipo String: metodi (2)

Metodi introdotti più recentemente [ES6]

- Il metodo `startsWith()` [`endsWith()`] restituisce **true** se una stringa inizia [termina] con un valore specificato, altrimenti **false** (case sensitive)
 - `const text = "Hello world, welcome to the universe.";`
 - `text.startsWith("Hello")` // restituisce true
 - `text.startsWith("hello")` // cosa restituisce?
 - `text.endsWith("universe.")` // restituisce true
- Un secondo parametro opzionale indica una posizione da cui iniziare [a cui terminare]
 - `text.startsWith("world", 6)` // controlla dalla posizione 6, restituisce true
 - `text.endsWith("world", 11)` // controlla i primi 11 caratteri, restituisce true

RICORDA: JavaScript conta le posizioni partendo da zero.

https://www.w3schools.com/js/js_string_search.asp

Il tipo String: metodi (3)

Metodi introdotti più recentemente [ES6]

- Il metodo `slice()` estrae una porzione di una stringa e restituisce la parte estratta in una nuova stringa.

- Accetta 1 o 2 parametri: la posizione iniziale e la posizione finale (esclusa) [opzionale]

```
1.const str = "Apple, Banana, Kiwi";  
2.let pos = str.slice(7, 13); // Banana  
3.pos = str.slice(7); // Banana, Kiwi
```

SEZIONE 3

Array e classi

Metodi per manipolare gli array (1)

1. `const fruits = ["Banana", "Orange", "Apple", "Mango"];`

Aggiungere elementi

- Il modo più semplice per aggiungere un nuovo elemento a un array è usare il metodo `push()`:

2. `fruits.push("Lemon");` // aggiunge un nuovo elemento ("Lemon")

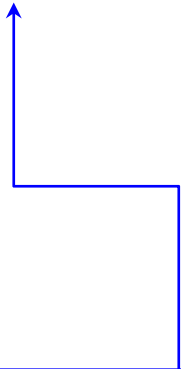
3. `let x = fruits.push();` // restituisce length: il valore di x è 5

Rimuovere l'ultimo elemento (pop)

- Il metodo `pop()` rimuove l'ultimo elemento da un array:

4. `fruits.pop();` // rimuove l'ultimo elemento ("Lemon")

5. `let y = fruits.pop();` // il valore di y è "Lemon"



Nota: gli elementi di array dichiarati con `const` POSSONO essere modificati

Metodi per manipolare gli array (2)

1. `const fruits = ["Banana", "Orange", "Apple", "Mango"];`

Rimuovere il primo elemento (shift)

- Il metodo `shift()` rimuove il primo elemento dell'array e fa scorrere tutti gli altri elementi a un indice inferiore:

6. `fruits.shift(); // rimuove il primo elemento ("Banana")`

7. `let z = fruits.shift(); // il valore di z è "Banana"`

Aggiungere in testa (unshift)

- Il metodo `unshift()` aggiunge un nuovo elemento all'array (in testa) e sposta in avanti gli elementi esistenti:

6. `fruits.unshift("Banana"); // aggiunge "Banana" in prima posizione`

Nota: gli elementi di array dichiarati con `const` POSSONO essere modificati

Metodi per manipolare gli array (3)

Splice di un array

- Il metodo splice() può essere usato per inserire nuovi elementi in un array:

1. `const fruits = ["Banana", "Orange", "Apple", "Mango"];`

2. `fruits.splice(2, 0, "Lemon", "Kiwi");`

```
// fruits = ["Banana", "Orange", "Lemon", "Kiwi", "Apple", "Mango"]
```

- Il primo parametro (2) indica la **posizione** in cui i nuovi elementi devono essere **aggiunti** (inseriti).
- Il secondo parametro (0) definisce **quanti** elementi devono essere **rimossi**.
- I parametri successivi ("Lemon", "Kiwi") definiscono i nuovi elementi da **aggiungere**.

- Il metodo splice() restituisce un array con gli elementi rimossi:

3. `let removed = fruits.splice(2, 2, "Lemon", "Kiwi");`

```
// fruits = ["Banana", "Orange", "Lemon", "Kiwi"]
```

```
// removed = ["Apple", "Mango"]
```

Le classi in JavaScript

ES6 ha introdotto le **classi**: una sintassi che permette di definire dei tipi di oggetti e creare istanze con `new`.

In JavaScript **tutti i valori sono oggetti**, tranne i tipi primitivi (string, number, boolean, null, undefined).

Quando scriviamo un letterale stiamo creando l'istanza di una classe predefinita. Per esempio:

```
const names = ["Anna", "Diana"];
```

è equivalente a:

```
const names = new Array("Anna", "Diana");
```

Anatomia di una classe

```
class Car {  
  constructor(brand, year) {  
    this.brand = brand;  
    this.year = year;  
  }  
  
  age(currentYear) {  
    return currentYear - this.year;  
  }  
  
  toString() {  
    return `${this.brand} (${this.year})`;  
  }  
}  
  
// uso  
const c = new Car("Fiat", 2020);  
console.log(c.toString()); // Fiat (2020)  
console.log(c.age(2026)); // 6
```

Anatomia di una classe

constructor: viene eseguito da new e inizializza i campi tramite this.

Metodi: si scrivono direttamente nel corpo della classe, senza la parola function.

Campi: non si dichiarano a priori, nascono assegnando this.x = ... nel constructor.

Differenze da Java

- Niente tipi nelle firme
- Niente public/private (campi privati: prefisso #)
- Niente overloading: una sola firma per nome

Classi predefinite, ereditarietà, static

Classi predefinite

JS offre molte classi pronte: Array, Date, Map, Set, Promise, ecc.

Esempio: Date ammette quattro firme di costruzione:

```
new Date()  
new Date(year, month, day, ...)  
new Date(milliseconds)  
new Date(dateString)
```

Convenzione: i nomi delle classi iniziano con la lettera maiuscola.

Ereditarietà e static

```
// estende la classe Car  
class ElectricCar extends Car {  
  constructor(brand, year, kWh) {  
    super(brand, year); // chiama parent  
    this.kWh = kWh;  
  }  
  
  static fromBrand(b) {  
    return new ElectricCar(b, 2026, 75);  
  }  
}  
  
const t = ElectricCar.fromBrand("Tesla");
```

- extends: ereditarietà singola, come in Java.
- super(...) nel constructor è obbligatorio.
- static: metodo della classe, non dell'istanza.

Iterazione su variabili multivalore

Iterazione su variabili multivalore

L'istruzione **for/of** considera tutti i valori di una struttura iterativa (array, stringhe, ...)

```
1.  const numbers = [1, 2, 3, 4, 5, 6];
2.  let sum = 0;

3.  for (let x of numbers) {
4.    sum = sum + x;
5.  }

6.  // sum vale 21
```

L'istruzione **for/in** considera tutte le proprietà di un oggetto

```
1.  const person = {fname:"John", lname:"Doe", age:25};
2.  let str = "";

3.  for (let x in person) {
4.    str += x + ": " + person[x] + "<br>";
5.  }

6.  // str: "fname: John<br>lname: Doe<br>age: 25<br>"
```

In entrambi gli esempi la variabile *x* contiene il valore corrente dell'indice.

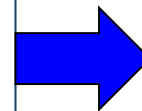
SEZIONE 4

Esercizio: DOM e CSS

Esercizio: oggetti e visualizzazione CSS

Dato un elenco di clienti, visualizzarlo in una tabella ben formattata.

```
[{Name: "Alfreds Futterkiste", City: "Berlin", Country: "Germany"}, {Name: "Berglunds snabbköp", City: "Luleå", Country: "Sweden"}, {Name: "Centro comercial Moctezuma", City: "México D.F.", Country: "Mexico"}, {Name: "Ernst Handel", City: "Graz", Country: "Austria"}, {Name: "FISSA Fabrica Inter. Salchichas S.A.", City: "Madrid", Country: "Spain"}, {Name: "Galería del gastrónomo", City: "Barcelona", Country: "Spain"}, {Name: "Island Trading", City: "Cowes", Country: "UK"}, {Name: "Königlich Essen", City: "Brandenburg", Country: "Germany"}, {Name: "Laughing Bacchus Wine Cellars", City: "Vancouver", Country: "Canada"}, {Name: "Magazzini Alimentari Riuniti", City: "Bergamo", Country: "Italy"}, {Name: "North/South", City: "London", Country: "UK"}, {Name: "Paris spécialités", City: "Paris", Country: "France"}, {Name: "Rattlesnake Canyon Grocery", City: "Albuquerque", Country: "USA"}, {Name: "Simons bistro", City: "København", Country: "Denmark"}, {Name: "The Big Cheese", City: "Portland", Country: "USA"}, {Name: "Vaffeljernet", City: "Århus", Country: "Denmark"}, {Name: "Wolski Zajazd", City: "Warszawa", Country: "Poland"}]
```



Customers

Alfreds Futterkiste	Berlin	Germany
Berglunds snabbköp	Luleå	Sweden
Centro comercial Moctezuma	México D.F.	Mexico
Ernst Handel	Graz	Austria
FISSA Fabrica Inter. Salchichas S.A.	Madrid	Spain
Galería del gastrónomo	Barcelona	Spain
Island Trading	Cowes	UK
Königlich Essen	Brandenburg	Germany
Laughing Bacchus Wine Cellars	Vancouver	Canada
Magazzini Alimentari Riuniti	Bergamo	Italy
North/South	London	UK
Paris spécialités	Paris	France
Rattlesnake Canyon Grocery	Albuquerque	USA
Simons bistro	København	Denmark
The Big Cheese	Portland	USA
Vaffeljernet	Århus	Denmark
Wolski Zajazd	Warszawa	Poland

... crea la tabella HTML, e ...

```
...
23. <body>

24. <h1>Customers</h1>
25. <div id="id01"></div>

26. <script>
27.     const list = [ { Name : "Alfreds Futterkiste", City : "Berlin", Country : "Germany" },
28.         { Name : "Berglunds snabbköp", City : "Luleå", Country : "Sweden" },
29.         { Name : "Centro comercial Moctezuma", "City" : "México D.F.", Country : "Mexico" },
30.         { Name : "Ernst Handel", City : "Graz", Country : "Austria" },
31.         { Name : "FISSA Fabrica Inter", City : "Madrid", Country : "Spain" },
32.         { Name : "Galería del gastrónomo", City : "Barcelona", Country : "Spain" } ];

33.     let out = "<table>";
34.     for (let customer of list) {
35.         out += "<tr><td>" + customer.Name + "</td><td>" + customer.City
36.             + "</td><td>" + customer.Country + "</td></tr>";
37.     }
38.     out += "</table>";
39.     document.getElementById("id01").innerHTML = out;
40. </script>
41. </body>
```

Cosa fa lo script

1. Definisce i dati

L'array list contiene 6 oggetti cliente con i campi Name, City, Country.

2. Costruisce l'HTML

Itera su ciascun cliente con for...of e concatena nella stringa out una riga <tr> con tre celle.

3. Inietta nel DOM

Scrive la tabella nel <div id="id01"> tramite innerHTML.

Esercizio: Aggiungi una riga di intestazione alla tabella; Inserisci un pulsante per visualizzare la tabella.

... crea la tabella HTML, e ...

```
...
23. <body>

24. <h1>Customers</h1>
25. <div id="id01"></div>

26. <script>
27.     const list = [ { Name : "Alfreds Futterkiste", City : "Berlin", Country : "Germany" },
28.         { Name : "Berglunds snabbköp", City : "Luleå", Country : "Sweden" },
29.         { Name : "Centro comercial Moctezuma", "City" : "México D.F.", Country : "Mexico" },
30.         { Name : "Ernst Handel", City : "Graz", Country : "Austria" },
31.         { Name : "FISSA Fabrica Inter", City : "Madrid", Country : "Spain" },
32.         { Name : "Galería del gastrónomo", City : "Barcelona", Country : "Spain" } ];

33.     let out = "<table>";
34.     for (let customer of list) {
35.         out += "<tr><td>" + customer.Name + "</td><td>" + customer.City
36.             + "</td><td>" + customer.Country + "</td></tr>";
37.     }
38.     out += "</table>";
39.     document.getElementById("id01").innerHTML = out;
40. </script>
41. </body>
```

Customers

Alfreds Futterkiste	Berlin	Germany
Berglunds snabbköp	Luleå	Sweden
Centro comercial Moctezuma	México D.F.	Mexico
Ernst Handel	Graz	Austria
FISSA Fabrica Inter. Salchichas S.A.	Madrid	Spain
Galería del gastrónomo	Barcelona	Spain
Island Trading	Cowes	UK
Königlich Essen	Brandenburg	Germany
Laughing Bacchus Wine Cellars	Vancouver	Canada
Magazzini Alimentari Riuniti	Bergamo	Italy
North/South	London	UK
Paris spécialités	Paris	France
Rattlesnake Canyon Grocery	Albuquerque	USA
Simons bistro	København	Denmark
The Big Cheese	Portland	USA
Vaffeljernet	Århus	Denmark
Wolski Zajazd	Warszawa	Poland

Esercizio: Aggiungi una riga di intestazione alla tabella; Inserisci un pulsante per visualizzare la tabella.

... aggiungi la formattazione con CSS

```
1. <html>
2. <head>
3. <title>List of customers</title>
4. <style>
5. h1 {
6.     border-bottom: 3px solid #cc9900;
7.     color: #996600;
8.     font-size: 30px;
9. }
10. table, th, td {
11.     border: 1px solid grey;
12.     border-collapse: collapse;
13.     padding: 5px;
14. }
15. table tr:nth-child(odd) {
16.     background-color: #f1f1f1;
17. }
18. table tr:nth-child(even) {
19.     background-color: #ffffff;
20. }
21. </style>
22. </head>
```

...

Customers

Alfreds Futterkiste	Berlin	Germany
Berglunds snabbköp	Luleå	Sweden
Centro comercial Moctezuma	México D.F.	Mexico
Ernst Handel	Graz	Austria
FISSA Fabrica Inter. Salchichas S.A.	Madrid	Spain
Galería del gastrónomo	Barcelona	Spain
Island Trading	Cowes	UK
Königlich Essen	Brandenburg	Germany
Laughing Bacchus Wine Cellars	Vancouver	Canada
Magazzini Alimentari Riuniti	Bergamo	Italy
North/South	London	UK
Paris spécialités	Paris	France
Rattlesnake Canyon Grocery	Albuquerque	USA
Simons bistro	København	Denmark
The Big Cheese	Portland	USA
Vaffeljernet	Århus	Denmark
Wolski Zajazd	Warszawa	Poland

SEZIONE 5

Programmazione funzionale

Elementi di programmazione: forEach

JavaScript ES5 ha introdotto i metodi di iterazione sugli array per operare su ciascun elemento

- Il metodo `forEach()` invoca una funzione (una funzione di callback) una volta per ciascun elemento dell'array.

1. `const numbers = [45, 4, 9, 16, 25];`

2. `let txt = "";`

3. `numbers.forEach(myFunction);`

4. `function myFunction(value, index, array) {`

5. `txt += value + "<br>";`

6. `}`

- La funzione accetta 3 argomenti:

- Il valore dell'elemento
- L'indice dell'elemento
- L'array stesso

Nota: l'esempio usa solo il parametro `value`.
Gli altri due possono essere omessi.

Elementi di programmazione: map

JavaScript ES5 ha introdotto i metodi di iterazione sugli array per operare su ogni elemento

- Il metodo `map()` crea un nuovo array applicando una funzione a ciascun elemento dell'array.

- Non esegue la funzione sugli elementi privi di valore

- Non modifica l'array originale

1. `const numbers1 = [45, 4, 9, 16, 25];`

2. `const numbers2 = numbers1.map(myFunction);`

3. `function myFunction(value, index, array) { // moltiplica ogni valore per 2`

4. `return value * 2;`

5. `} // array risultante: [90, 8, 18, 32, 50]`

- La funzione accetta 3 argomenti:

- il valore dell'elemento

- l'indice dell'elemento

- l'array stesso

NOTA: quando una funzione di callback usa solo il parametro `value`, i parametri `index` e `array` possono essere omessi

Elementi di programmazione: filter

JavaScript ES5 ha introdotto metodi di iterazione sugli array per operare su ogni elemento

- Il metodo `filter()` crea un nuovo array contenente gli elementi che superano un test.

- Non esegue la funzione sugli elementi senza valore
- Non modifica l'array originale

1. `const numbers = [45, 4, 9, 16, 25];`

2. `const over18 = numbers.filter(myFunction);`

3. `function myFunction(value, index, array) {`

4. `// crea un nuovo array con gli elementi maggiori di 18`

5. `return value > 18;`

6. `} // array risultante: [45, 25]`

- La funzione accetta 3 argomenti:

- il valore dell'elemento
- l'indice dell'elemento
- l'array stesso

Nota: l'esempio usa solo il parametro `value`. Gli altri due possono essere omessi.

Elementi di programmazione: reduce

JavaScript ES5 ha introdotto i metodi di iterazione sugli array per operare su ogni elemento

- Il metodo `reduce()` esegue una funzione su ogni elemento dell'array per produrre un singolo valore.

- Lavora da sinistra a destra sull'array

- Non modifica l'array originale

- `const numbers = [45, 4, 9, 16, 25];`

- `let sum = numbers.reduce(myFunction);`

- `function myFunction(total, value, index, array) { // somma tutti i numeri di un array`

- `return total + value;`

- `} // valore risultante: 99`

- La funzione accetta 4 argomenti:

- L'accumulatore (il valore iniziale o il valore restituito in precedenza)

- Il valore corrente

- L'indice corrente

- L'array stesso

Il metodo `reduce()` può accettare un valore iniziale.

```
2. let sum = numbers.reduce(myFunction, 100);
```

Se il valore iniziale non è dichiarato, è `array[0]`

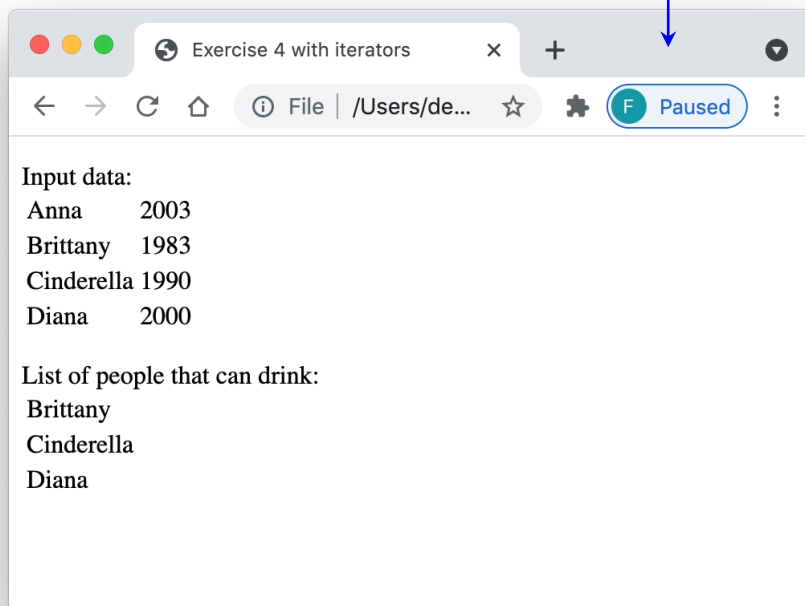
Un esempio con oggetti

Dato un array di oggetti persona

Verificare chi ha raggiunto la maggiore età utilizzando il metodo `filter()`

Una possibile struttura della soluzione è riportata a fianco

Una possibile visualizzazione



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Exercise 4 with iterators</title>
</head>
<body>
<p>Input data: <span id="input"></span></p>
<p>List of people that can drink:
<span id="output"></span></p>

<script>

  const date = new Date();
  const thisYear = date.getFullYear();
  const people = [{name: "Anna", birthYear: 2003},
    {name: "Brittany", birthYear: 1983},
    {name: "Cinderella", birthYear: 1990},
    {name: "Diana", birthYear: 2000}];

  ...
</script>

</body>
</html>
```


Un esempio con oggetti

```
1. <!DOCTYPE html>
2. <html lang="en">
3. <head>
4.   <meta charset="UTF-8">
5.   <title>Exercise 4 with iterators</title>
6. </head>
7. <body>
8. <p>Input data: <span id="input"></span></p>
9. <p>List of people that can drink:
10. <span id="output"></span></p>

11. <script>
12.   function computeAge(thisYear, birthYear) {
13.     let age = thisYear - birthYear; // compute age
14.     return age;
15.   }

16.   function myFunction(value) {
17.     return computeAge(thisYear, value.birthYear) > 18;
18.   }

19.   const date = new Date();
20.   const thisYear = date.getFullYear();
21.   const people = [{name: "Anna", birthYear: 2003},
22.     {name: "Brittany", birthYear: 1983},
23.     {name: "Cinderella", birthYear: 1990},
24.     {name: "Diana", birthYear: 2000}];
```

Cosa fa il codice

Vogliamo mostrare l'elenco completo di un gruppo di persone e poi, separatamente, solo quelle **maggioresni**.

1. Le funzioni di supporto

computeAge calcola l'età come differenza fra anno corrente e anno di nascita.

myFunction è il **predicato** che useremo con filter: restituisce true se la persona ha più di 18 anni.

2. I dati

Si calcola l'anno corrente con new Date() e si definisce un array people di 4 oggetti con name e birthYear.

Continua nella slide successiva →

Un esempio con oggetti

Cosa fa il codice

← Segue dalla slide precedente

3. Tabella di tutte le persone

Doppio ciclo annidato: for...of sulle persone (valori) e for...in sui campi di ciascun oggetto (chiavi). Il risultato viene scritto nello span#input.

4. Filtro dei maggiorenni

people.filter(myFunction) restituisce un nuovo array canDrink con i soli oggetti per cui il predicato vale true.

5. Tabella dei filtrati

Si itera su canDrink con un for classico, mostrando solo il name nello span#output.

```
25. let out = "<table>";
26. for (let person of people) {
27.   out += "<tr>";
28.   for (let field in person) {
29.     out += "<td>" + person[field] + "</td>";
30.   }
31.   out += "</tr>";
32. }
33. out += "</table>";
34. document.getElementById("input").innerHTML = out;

35. let canDrink = people.filter(myFunction);

36. out = "<table>";
37. for (let i = 0; i < canDrink.length; i++) {
38.   out += "<tr>";
39.   out += "<td>" + canDrink[i].name + "</td>";
40.   out += "</tr>";
41. }
42. out += "</table>";
43. document.getElementById("output").innerHTML = out;

44. </script>

45. </body>
46. </html>
```